



SAPIENZA
UNIVERSITÀ DI ROMA

Practical Network Defense a.y. 23-24

Assignment 1 Report - ACME 27 Group

Mija Simone Nicolae, 1895592 - mija.1895592@studenti.uniroma1.it

Palozzi Massimo, 2132174 - palozzi.2132174@studenti.uniroma1.it

Scarselli Ilaria, 1918975 - scarselli.1918975@studenti.uniroma1.it

Uffreduzzi Giorgia, 1668798 - uffreduzzi.1668798@studenti.uniroma1.it

October 23, 2024



Contents

1	Introduction	3
2	Security Policy Enforcement	3
2.1	Task 1 - All the ACME hosts must use the internal DNS Server as a DNS resolver . . .	3
2.2	Task 2 - The HTTP/HTTPS service provided in the DMZ has to be accessible from the Internet	4
2.3	Task 3 - The proxy service provided in the DMZ has to be accessible by the hosts of the ACME network and from the Internet	5
2.4	Task 4 - Besides the DNS resolver, the other services in the Internal server network must be accessible only to hosts of the Client and DMZ networks	7
2.5	Task 5 - All the hosts (but the Client network hosts) have to use the syslog and the log collector services on the Log server (syslog) and Graylog server	8
2.5.1	Logserver	8
2.5.2	Graylog	8
2.5.3	Clients configuration	8
2.6	Task 6 - The Greenbone server must be able to scan all the network hosts	9
2.7	Task 7 - All network hosts must be managed via SSH only from hosts within the Client network	10
2.8	Task 8 - The Client network hosts have only access to external web services (HTTP/HTTPS) through the proxy server in the DMZ	10
2.9	Task 9 - Any packet the Main Firewall receives on port 65432 should be redirected to port 80 of the proxy host	11
2.10	Task 10 - All the internal hosts should use the public IP address of the Main Firewall to exit towards the Internet	12
2.11	Task 11 - All the hosts of the ACME network should be able to ping (and receive replies of) the other hosts and the Internet hosts	13
2.12	Task 12 - Only hosts in the DMZ should be reachable using the ping and traceroute tools from the Internet	13
2.13	Task 13 - ICMP redirect packets should not cross any network	14
2.14	Task 14 - Anything that is not explicitly allowed has to be denied	14
3	Conclusions	15



1 Introduction

This report outlines the implementation of security policies in the ACME network. Initially, we aimed to divide our tasks as much as possible in order to work in parallel. To facilitate easier connections, we enabled our VPN IP addresses to connect directly to the firewalls and route all traffic to the hosts, later disabling this option to simulate an external connection. We chose to implement the entire ACME network using both IPv6 and IPv4, configuring all the necessary addresses, and we decided that all ACME hosts should use stable-privacy IPv6 addresses. Additionally, we configured several services, including a transparent proxy.

2 Security Policy Enforcement

2.1 Task 1 - All the ACME hosts must use the internal DNS Server as a DNS resolver

Initial brainstorming - To meet the task's requirement, we need to allow IPv4 and IPv6 traffic to enable all hosts in the ACME network to query and make requests to the DNS.

Evaluation of the security policy - The DNS must be able to resolve the hosts within the ACME network and serve as the primary (and only) DNS for these hosts. However, since the hosts can also access the internet, our DNS server needs to reference an external DNS to handle external queries on behalf of the hosts and provide the results.

Policy implementation in OPNsense - After configuring the DNS and modifying its */etc/hosts* file, we also added references to the external Sapienza DNS servers, which were present by default, to its */etc/resolv.conf*. For all other hosts, we only added references to the internal DNS server of the ACME network and prevented these files from being modified on reboot. Lastly, we applied rules on every interface except the WAN to allow both IPv4 and IPv6 traffic on port 53 to the DNSserver, enabling requests to it.

Test of the configuration - We tested the DNSserver from all hosts using the **host** and **ping** commands, using also IPv6. Additionally, we tested it for external resolutions as well:

```
root@arpwatch-clients:~# host webserver
webserver has address 100.100.6.2
webserver has IPv6 address 2001:470:b5b8:1b06:a65e:ad2c:6d4c:3d75
root@arpwatch-clients:~# host www.google.it
www.google.it has address 216.58.209.35
www.google.it has IPv6 address 2a00:1450:4002:809::2003
```



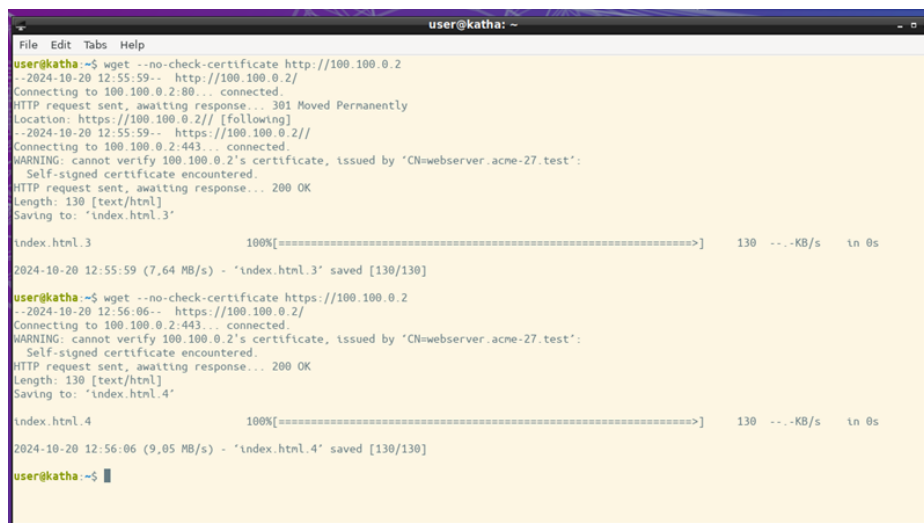
2.2 Task 2 - The HTTP/HTTPS service provided in the DMZ has to be accessible from the Internet

Initial brainstorming - The web server with the private address 100.100.6.2 is located in the DMZ and provides HTTP and HTTPS services. It is accessible to external users via the public IP address 100.100.0.2. This setup is essential for ensuring that users can reach the server from the internet.

Evaluation of the security policy - The firewall rules have been configured to allow incoming traffic specifically on ports 80 and 443. This implementation is essential for maintaining a balance between security and accessibility, ensuring that authorized users can access the web services while minimizing exposure to potential threats. Additionally, it is important to address the requirement for a valid SSL certificate to improve security for HTTPS traffic.

Policy implementation in OPNsense - To enable access to the web server, Network Address Translation (NAT) is employed. This process translates the public address 100.100.0.2 to the private address 100.100.6.2 for traffic originating from the internet. Specific ports are designated for this access: port 80 for HTTP and port 443 for HTTPS. The firewall configuration on the WAN interface is set to permit incoming traffic directly to these ports, guaranteeing that requests targeting the public IP address are processed correctly.

Test of the configuration - To validate the configuration a test is performed from an external network identified by the address range 100.101.0.0/24. During testing, a request was made to the HTTP service at the public IP address 100.100.0.2.



```
user@katha: ~  
File Edit Tabs Help  
user@katha:~$ wget --no-check-certificate http://100.100.0.2  
--2024-10-20 12:55:59-- http://100.100.0.2/  
Connecting to 100.100.0.2:80... connected.  
HTTP request sent, awaiting response... 301 Moved Permanently  
Location: https://100.100.0.2/ [following]  
--2024-10-20 12:55:59-- https://100.100.0.2/  
Connecting to 100.100.0.2:443... connected.  
WARNING: cannot verify 100.100.0.2's certificate, issued by 'CN=webserver.acme-27.test':  
Self-signed certificate encountered.  
HTTP request sent, awaiting response... 200 OK  
Length: 130 [text/html]  
Saving to: 'index.html.3'  
  
index.html.3      100%[=====] 130 --KB/s  in 0s  
2024-10-20 12:55:59 (7.64 MB/s) - 'index.html.3' saved [130/130]  
  
user@katha:~$ wget --no-check-certificate https://100.100.0.2  
--2024-10-20 12:56:06-- https://100.100.0.2/  
Connecting to 100.100.0.2:443... connected.  
WARNING: cannot verify 100.100.0.2's certificate, issued by 'CN=webserver.acme-27.test':  
Self-signed certificate encountered.  
HTTP request sent, awaiting response... 200 OK  
Length: 130 [text/html]  
Saving to: 'index.html.4'  
  
index.html.4      100%[=====] 130 --KB/s  in 0s  
2024-10-20 12:56:06 (9.05 MB/s) - 'index.html.4' saved [130/130]  
  
user@katha:~$
```

The server responded with a 301 Moved Permanently status, indicating a redirect to the HTTPS version of the service. Although the connection was successfully established, a warning regarding a self-signed certificate was issued, which was bypassed using the `--no-check-certificate` option. A subsequent request



to the HTTPS service confirmed the connection on port 443, with the server responding with a 200 OK status. However, the self-signed certificate raised a warning, highlighting the need for a valid certificate from a recognized Certificate Authority to ensure secure communication without user alerts.

2.3 Task 3 - The proxy service provided in the DMZ has to be accessible by the hosts of the ACME network and from the Internet

Initial brainstorming - In the DMZ, a Squid-based proxy service has been implemented to efficiently manage network traffic. This open-source proxy server operates in two distinct modes: a **forward proxy** listening on port 3127 for HTTP connections and on port 3126 for HTTPS connections, allowing external hosts (Internet and External network hosts) to securely access internal services. Additionally, there is a **transparent proxy** on ports 3128 and 3129, designed to automatically intercept HTTP and HTTPS requests from internal hosts (internal clients and internal servers network hosts). We consider "external services" not only the Internet but also web services within ACME's external network, such as the Fantastic coffee machine. For "internal services," we refer to services within the servers network.

Evaluation of the security policy - Internet access is enabled by the public IP address 100.100.0.2, allowing external hosts to reach internal services securely. Specific firewall rules on the WAN interface permit incoming TCP traffic to this address. To enhance security, Network Address Translation (NAT) is employed to map the public IP to the internal proxy address (100.100.6.3). Hosts from the external network (100.100.4.0), although part of the ACME network, utilize the private proxy address to access services, effectively bypassing the public IP. When an external host seeks to connect to an internal service or the Internet, it sends a request to the proxy on the specified ports. Those using the proxy in forward mode are required to configure it manually, specifying the correct proxy address and port. The proxy processes these requests and forwards them to either the appropriate internal services or external destinations, ensuring secure access. For internal hosts, the proxy operates in transparent mode, seamlessly intercepting HTTP and HTTPS requests without requiring manual configuration on client devices. This dual-mode operation enhances overall security by ensuring that internal traffic remains protected while also allowing for controlled external access.

Policy implementation in OPNsense - On the WAN interface, specific firewall rules have been configured to allow incoming traffic to the proxy. Internet hosts can securely access internal services through Network Address Translation (NAT), which translates the public IP to the private address of the proxy. This setup ensures that requests from external sources are properly routed to the internal services without exposing the internal network structure. For the EXTERNAL interface, firewall rules have been established to allow outbound traffic to the proxy, facilitating access for external clients to the ACME network. Traffic on ports 3127 and 3126 is routed to the proxy, ensuring that HTTP and HTTPS requests are handled securely. Users must manually configure the proxy, specifying the appropriate address and port. Additionally, Policy-Based Routing (PBR) has been implemented on



the INTERNAL interface. This configuration automatically redirects requests from internal hosts destined for standard HTTP (80) and HTTPS (443) ports to the appropriate proxy ports. This setup ensures that all relevant traffic is routed through the transparent proxy, guaranteeing that requests are intercepted and managed effectively without the need for manual configuration on the clients.

Test of the configuration - To validate the setup, a series of tests were done. First, access to the proxy from an external network (100.101.0.0/24) was attempted by connecting to 100.100.0.2:3127. This confirmed successful external accessibility. Furthermore, a test was performed to access the Graylog server located at 100.100.1.10 using the proxy, ensuring that the internal routing functions:

```
user@katha: ~  
File Edit Tabs Help  
user@katha:~$ wget -e use_proxy=yes -e http_proxy=http://100.100.0.2:3127 http://100.100.1.10  
--2024-10-20 14:58:31-- http://100.100.1.10/  
Connecting to 100.100.0.2:3127... connected.  
Proxy request sent, awaiting response... 200 OK  
Length: 1023 [text/html]  
Saving to: 'index.html.7'  
  
index.html.7 100%[=====] 1023 --.-KB/s in 0.08s  
2024-10-20 14:58:33 (13.0 KB/s) - 'index.html.7' saved [1023/1023]  
  
user@katha:~$
```

For internal hosts on the network 100.100.2.0/24, tests confirmed that HTTP and HTTPS requests were successfully intercepted by the proxy.

```
user@pci: ~  
File Edit Tabs Help  
user@katha:~$ ssh 100.100.2.100  
user@100.100.2.100's password:  
Linux pci 6.6.9-and64 #1 SMP PREEMPT_DYNAMIC Kali 6.6.9-1kali1 (2024-01-08) x86_64  
  
The programs included with the Kali GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/*copyright.  
  
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Sun Oct 20 19:51:11 2024 from 100.101.0.3  
(user@pci) ~  
$ wget http://100.101.0.3  
--2024-10-20 19:51:42-- http://100.101.0.3/  
Connecting to 100.101.0.3:80... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 130 [text/html]  
Saving to: 'index.html.12'  
  
index.html.12 100%[=====] 130 --.-KB/s in 0s  
2024-10-20 19:51:42 (20.4 MB/s) - 'index.html.12' saved [130/130]  
  
(user@pci) ~  
$
```


positive, confirming that these hosts could establish connections. Conversely, we also tested access attempts from the External and Internet networks. These attempts were effectively redirected to the forward proxy, confirming that our security measures were functioning as intended. By blocking direct access to the servers from these networks, we successfully maintained the integrity of our internal resources.

2.5 Task 5 - All the hosts (but the Client network hosts) have to use the syslog and the log collector services on the Log server (syslog) and Graylog server

Initial brainstorming - To provide log server and Graylog server use, it was necessary to configure them.

2.5.1 Logserver

The rsyslog was configured by editing the `/etc/rsyslog.conf` file. To allow the server to listen the UDP port 514, the following commands were added:

```
1 module (load="imudp")
2 input (type="imudp" port="514")
```

2.5.2 Graylog

The graylog server was configured using the web interface. To allow the server to listen the UDP port 514, we had to configure the Log Inputs in Graylog.

- In the Graylog web interface, System *i* Inputs and selected the Syslog UDP.
- Configured the input to listen on the appropriate port. In our case, the default port 514 for Syslog.
- After starting the input, Graylog server will be ready to receive logs.

2.5.3 Clients configuration

To send clients logs to graylog and logserver, we used and configured rsyslog for each clients by adding the following command on file `/etc/rsyslog.conf`

```
1 *.info action(type="omfwd" target="100.100.1.10" port="514" protocol="udp")
2 *.info action(type="omfwd" target="100.100.1.10" port="514" protocol="udp")
```

Evaluation of the security policy - All hosts but clients network have to send logs to graylog and logserver.

Policy implementation in OPNsense - Allow all UDP traffic on port 514 with destination graylog or logserver.

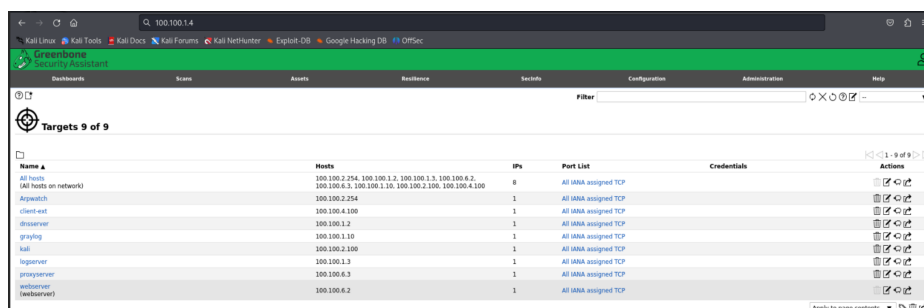
Test of the configuration - To be sure that all hosts send correctly logs to graylog and logserver, we used the following command to generate a simple test log:

```
1 logger -p user.warn "Warning log - Test"
```

We confirmed that each host send correctly the logs by looking logserver file /var/log/syslog and checking graylog logs on its web interface.

2.6 Task 6 - The Greenbone server must be able to scan all the network hosts

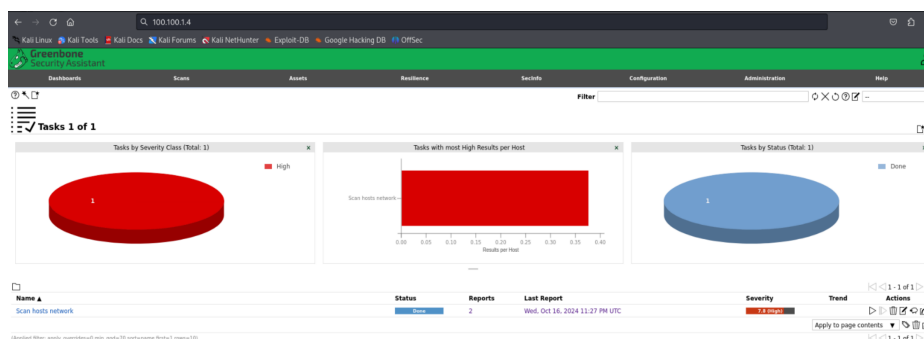
Initial brainstorming - To allow all hosts to be scanned, we configured each one by adding them as target using the greenbone web interface.



Name	Hosts	IPs	Port List	Credentials	Actions
All hosts (All hosts on network)	100.100.1.254, 100.100.1.2, 100.100.1.3, 100.100.6.2, 100.100.6.3, 100.100.1.10, 100.100.2.100, 100.100.4.100	8	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]
Arguswitch	100.100.2.254	1	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]
client-wifi	100.100.4.100	1	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]
logserver	100.100.1.2	1	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]
graylog	100.100.1.10	1	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]
kali	100.100.2.100	1	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]
logserver	100.100.1.3	1	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]
grayserver	100.100.6.3	1	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]
webserver (webserver)	100.100.6.2	1	All NMAP assigned TCP		[Edit] [Delete] [Refresh] [Add]

After declaring all targets, we created a new scan, using once again the web interface.

Test of the configuration - We launched a scan on all network hosts.



Greenbone successfully created a report and marked our proxy (squid) not secure. The ACME Industry will fix this by creating a new proxy written in C (they will create another mess).



2.7 Task 7 - All network hosts must be managed via SSH only from hosts within the Client network

Initial brainstorming - In order to provide easier access to the ACME hosts, we decided to allow our IP addresses (*100.101.0.0/24*) to access all the machines. The same has been done for the Internal clients subnet as well.

Evaluation of the security policy - Only hosts within the internal clients subnet should be able to use SSH to access other hosts. It was necessary to add the following line to each host to allow password-based login:

```
1 echo 'PermitRootLogin yes' >> /etc/ssh/sshd_config; systemctl restart ssh
```

Policy implementation in OPNsense - We have allowed SSH traffic on the WAN interface only for our IP addresses. Additionally, we have allowed outbound SSH traffic from the clients subnet and our IP addresses on the DMZ, External Clients, Internal Servers, and Internal Clients interfaces. All other SSH connections have been denied.

Test of the configuration - To test the configuration, we tried connecting using SSH from every host to every other host. Kali and ARPWatch were correctly able to do so, while others could not.

2.8 Task 8 - The Client network hosts have only access to external web services (HTTP/HTTPS) through the proxy server in the DMZ

Initial brainstorming - As previously discussed, external web services include both the Internet and the Fantastic Coffee service, which operates exclusively on port 80 within the external service network. From this specification, we have deduced that HTTP and HTTPS connections to the web server in the DMZ should not be redirected to the proxy.

Evaluation of the security policy - The security policy must ensure that clients can access external web services without unnecessary routing through the proxy. However, connections to the Fantastic Coffee service and HTTP/HTTPS requests to the Internet must necessarily pass through the transparent proxy to maintain security and control over web traffic.

Policy implementation in OPNsense - We utilized NAT to remap HTTP/HTTPS traffic addressed to the DMZ web server, allowing for direct communication from internal clients without routing through the proxy. This setup ensures that traffic meant for the DMZ web server is handled directly. On the other hand, Policy-Based Routing (PBR) on the INTERNAL interface is employed to redirect other HTTP/HTTPS traffic, specifically requests to the Internet and the Fantastic Coffee service, to the proxy for interception.

Test of the configuration - Tests are going to observe the interaction between web servers and proxy server.

```
File Edit Tabs Help

The programs included with the Kali GNU/Linux system are free software; the exact distribution terms for each program are described in the individual files in /usr/share/doc/*/.copyright
```

```
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent permitted by applicable law.
```

```
Last login: Tue Oct 22 09:42:22 2024 from 100.101.0.3  
[user@kali:~]$  
[~] user@kali ~$ curl -c check-certificates https://100.100.6.2/  
- 2024-10-22 09:44:47 - https://100.100.6.2/  
Connecting to 100.100.6.2:443... connected.  
WARNING: The certificate of "https://100.6.2/" is not trusted.  
WARNING: The certificate of "https://100.6.2/" doesn't have a known issuer.  
HTTP request sent, awaiting response... 200 OK  
length: 130 [text/html]  
Saving to: '.index.html'*
```

```
index.html.18      100K[=====] 130    --KB/s   in 0s  
  
2024-10-22 09:44:47 (129 MB/s) : '.index.html.18' saved [130/130]
```

```
[user@kali:~]$  
root@proxyserver/var/log/apache2# tcpdump port 3129  
tcpdump verbose output suppressed, use -v|-vv|--for full protocol decoded  
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
```

In the result proxy does not intercept traffic when client attempt to contact DMZ apache webserver. To verify if the traffic between kali host and external services is intercepted by proxy server, we try to access to fantasticcoffee machine from internal host:

The screenshot displays a Kali Linux terminal window with two panes. The left pane shows a Netcat listener on 192.168.0.10. It receives a connection from 10.10.10.10. The user enters 'nc' to interact with the connection. The right pane shows a Metasploit Meterpreter session on 10.10.10.10. The user enters 'run' to execute a command, which triggers a series of HTTP requests from the victim machine to the listener's IP address. The requests are for various paths like /api/, /api/1.0/, /api/1.0.1/, /api/1.0.2/, /api/1.0.3/, /api/1.0.4/, /api/1.0.5/, /api/1.0.6/, /api/1.0.7/, /api/1.0.8/, /api/1.0.9/, /api/1.0.10/, /api/1.0.11/, /api/1.0.12/, /api/1.0.13/, /api/1.0.14/, /api/1.0.15/, /api/1.0.16/, /api/1.0.17/, /api/1.0.18/, /api/1.0.19/, /api/1.0.20/, /api/1.0.21/, /api/1.0.22/, /api/1.0.23/, /api/1.0.24/, /api/1.0.25/, /api/1.0.26/, /api/1.0.27/, /api/1.0.28/, /api/1.0.29/, /api/1.0.30/, /api/1.0.31/, /api/1.0.32/, /api/1.0.33/, /api/1.0.34/, /api/1.0.35/, /api/1.0.36/, /api/1.0.37/, /api/1.0.38/, /api/1.0.39/, /api/1.0.40/, /api/1.0.41/, /api/1.0.42/, /api/1.0.43/, /api/1.0.44/, /api/1.0.45/, /api/1.0.46/, /api/1.0.47/, /api/1.0.48/, /api/1.0.49/, /api/1.0.50/, /api/1.0.51/, /api/1.0.52/, /api/1.0.53/, /api/1.0.54/, /api/1.0.55/, /api/1.0.56/, /api/1.0.57/, /api/1.0.58/, /api/1.0.59/, /api/1.0.60/, /api/1.0.61/, /api/1.0.62/, /api/1.0.63/, /api/1.0.64/, /api/1.0.65/, /api/1.0.66/, /api/1.0.67/, /api/1.0.68/, /api/1.0.69/, /api/1.0.70/, /api/1.0.71/, /api/1.0.72/, /api/1.0.73/, /api/1.0.74/, /api/1.0.75/, /api/1.0.76/, /api/1.0.77/, /api/1.0.78/, /api/1.0.79/, /api/1.0.80/, /api/1.0.81/, /api/1.0.82/, /api/1.0.83/, /api/1.0.84/, /api/1.0.85/, /api/1.0.86/, /api/1.0.87/, /api/1.0.88/, /api/1.0.89/, /api/1.0.90/, /api/1.0.91/, /api/1.0.92/, /api/1.0.93/, /api/1.0.94/, /api/1.0.95/, /api/1.0.96/, /api/1.0.97/, /api/1.0.98/, /api/1.0.99/, /api/1.0.100/, /api/1.0.101/, /api/1.0.102/, /api/1.0.103/, /api/1.0.104/, /api/1.0.105/, /api/1.0.106/, /api/1.0.107/, /api/1.0.108/, /api/1.0.109/, /api/1.0.110/, /api/1.0.111/, /api/1.0.112/, /api/1.0.113/, /api/1.0.114/, /api/1.0.115/, /api/1.0.116/, /api/1.0.117/, /api/1.0.118/, /api/1.0.119/, /api/1.0.120/, /api/1.0.121/, /api/1.0.122/, /api/1.0.123/, /api/1.0.124/, /api/1.0.125/, /api/1.0.126/, /api/1.0.127/, /api/1.0.128/, /api/1.0.129/, /api/1.0.130/, /api/1.0.131/, /api/1.0.132/, /api/1.0.133/, /api/1.0.134/, /api/1.0.135/, /api/1.0.136/, /api/1.0.137/, /api/1.0.138/, /api/1.0.139/, /api/1.0.140/, /api/1.0.141/, /api/1.0.142/, /api/1.0.143/, /api/1.0.144/, /api/1.0.145/, /api/1.0.146/, /api/1.0.147/, /api/1.0.148/, /api/1.0.149/, /api/1.0.150/, /api/1.0.151/, /api/1.0.152/, /api/1.0.153/, /api/1.0.154/, /api/1.0.155/, /api/1.0.156/, /api/1.0.157/, /api/1.0.158/, /api/1.0.159/, /api/1.0.160/, /api/1.0.161/, /api/1.0.162/, /api/1.0.163/, /api/1.0.164/, /api/1.0.165/, /api/1.0.166/, /api/1.0.167/, /api/1.0.168/, /api/1.0.169/, /api/1.0.170/, /api/1.0.171/, /api/1.0.172/, /api/1.0.173/, /api/1.0.174/, /api/1.0.175/, /api/1.0.176/, /api/1.0.177/, /api/1.0.178/, /api/1.0.179/, /api/1.0.180/, /api/1.0.181/, /api/1.0.182/, /api/1.0.183/, /api/1.0.184/, /api/1.0.185/, /api/1.0.186/, /api/1.0.187/, /api/1.0.188/, /api/1.0.189/, /api/1.0.190/, /api/1.0.191/, /api/1.0.192/, /api/1.0.193/, /api/1.0.194/, /api/1.0.195/, /api/1.0.196/, /api/1.0.197/, /api/1.0.198/, /api/1.0.199/, /api/1.0.200/, /api/1.0.201/, /api/1.0.202/, /api/1.0.203/, /api/1.0.204/, /api/1.0.205/, /api/1.0.206/, /api/1.0.207/, /api/1.0.208/, /api/1.0.209/, /api/1.0.210/, /api/1.0.211/, /api/1.0.212/, /api/1.0.213/, /api/1.0.214/, /api/1.0.215/, /api/1.0.216/, /api/1.0.217/, /api/1.0.218/, /api/1.0.219/, /api/1.0.220/, /api/1.0.221/, /api/1.0.222/, /api/1.0.223/, /api/1.0.224/, /api/1.0.225/, /api/1.0.226/, /api/1.0.227/, /api/1.0.228/, /api/1.0.229/, /api/1.0.230/, /api/1.0.231/, /api/1.0.232/, /api/1.0.233/, /api/1.0.234/, /api/1.0.235/, /api/1.0.236/, /api/1.0.237/, /api/1.0.238/, /api/1.0.239/, /api/1.0.240/, /api/1.0.241/, /api/1.0.242/, /api/1.0.243/, /api/1.0.244/, /api/1.0.245/, /api/1.0.246/, /api/1.0.247/, /api/1.0.248/, /api/1.0.249/, /api/1.0.250/, /api/1.0.251/, /api/1.0.252/, /api/1.0.253/, /api/1.0.254/, /api/1.0.255/, /api/1.0.256/, /api/1.0.257/, /api/1.0.258/, /api/1.0.259/, /api/1.0.260/, /api/1.0.261/, /api/1.0.262/, /api/1.0.263/, /api/1.0.264/, /api/1.0.265/, /api/1.0.266/, /api/1.0.267/, /api/1.0.268/, /api/1.0.269/, /api/1.0.270/, /api/1.0.271/, /api/1.0.272/, /api/1.0.273/, /api/1.0.274/, /api/1.0.275/, /api/1.0.276/, /api/1.0.277/, /api/1.0.278/, /api/1.0.279/, /api/1.0.280/, /api/1.0.281/, /api/1.0.282/, /api/1.0.283/, /api/1.0.284/, /api/1.0.285/, /api/1.0.286/, /api/1.0.287/, /api/1.0.288/, /api/1.0.289/, /api/1.0.290/, /api/1.0.291/, /api/1.0.292/, /api/1.0.293/, /api/1.0.294/, /api/1.0.295/, /api/1.0.296/, /api/1.0.297/, /api/1.0.298/, /api/1.0.299/, /api/1.0.300/, /api/1.0.301/, /api/1.0.302/, /api/1.0.303/, /api/1.0.304/, /api/1.0.305/, /api/1.0.306/, /api/1.0.307/, /api/1.0.308/, /api/1.0.309/, /api/1.0.310/, /api/1.0.311/, /api/1.0.312/, /api/1.0.313/, /api/1.0.314/, /api/1.0.315/, /api/1.0.316/, /api/1.0.317/, /api/1.0.318/, /api/1.0.319/, /api/1.0.320/, /api/1.0.321/, /api/1.0.322/, /api/1.0.323/, /api/1.0.324/, /api/1.0.325/, /api/1.0.326/, /api/1.0.327/, /api/1.0.328/, /api/1.0.329/, /api/1.0.330/, /api/1.0.331/, /api/1.0.332/, /api/1.0.333/, /api/1.0.334/, /api/1.0.335/, /api/1.0.336/, /api/1.0.337/, /api/1.0.338/, /api/1.0.339/, /api/1.0.340/, /api/1.0.341/, /api/1.0.342/, /api/1.0.343/, /api/1.0.344/, /api/1.0.345/, /api/1.0.346/, /api/1.0.347/, /api/1.0.348/, /api/1.0.349/, /api/1.0.350/, /api/1.0.351/, /api/1.0.352/, /api/1.0.353/, /api/1.0.354/, /api/1.0.355/, /api/1.0.356/, /api/1.0.357/, /api/1.0.358/, /api/1

2.9 Task 9 - Any packet the Main Firewall receives on port 65432 should be redirected to port 80 of the proxy host

Initial brainstorming - Since the proxy host is in the DMZ, it should be reachable from the internet. Therefore, we allowed TCP/UDP traffic with destination port 65432 on every interfaces.

Evaluation of the security policy - Any packet with destination port 65432 that reaches one of the main firewall interfaces is to be forwarded to the proxy host on port 80.

Policy implementation in OPNsense - Allow all traffic received on port 65432 of the main firewall as a floating rule. Port forward every packet with destination address main-firewall and destination port 65432 to proxy host on port 80.

Test of the configuration - In order to test the configuration, we started a python server on port 80 on the proxy host and we made an http request to the main-firewall on port 65432 from other ACME hosts:

```
(user@pc1)~$ curl -s main-firewall:65432 > /dev/null

root@proxyserver:~# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
100.100.2.100 - - [12/Oct/2024 00:08:32] "GET / HTTP/1.1" 200 -
```

2.10 Task 10 - All the internal hosts should use the public IP address of the Main Firewall to exit towards the Internet

Initial brainstorming - To satisfy this requirement, it is necessary to use NAT. For IPv4 it is necessary to use the public IP address of the Main firewall to exit towards the internet, and thus every ACME host is already using NAT, while for IPv6 this is not necessary, and therefore we configured only the internal hosts to use the public IP address of the firewall. In our ACME configuration all internal hosts must use the transparent proxy to exit towards the internet: for this reason we had to enable NAT on ipv6 proxyserver too for http and https.

Evaluation of the security policy - Use NAT on WAN interface.

Policy implementation in OPNsense - Implement outbound NAT: for IPv4, every packet received on any interfaces to WAN; for IPv6, every packet received on the INTERNAL interface should use the IP address of the WAN interface to exit towards the internet. However, this was not possible, since the WAN does not receive a global address, and we decided to use the global address of the INTERNAL interface instead.

Test of the configuration - For IPv4, we started an http server on our host to check if the source of the requests is the IP address of the WAN interface of the main firewall:

```
(user@pc1)~$ curl -s 100.101.0.4 > /dev/null

PS C:\Users> python3 -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
::ffff:100.100.0.2 - - [14/Oct/2024 14:31:17] "GET / HTTP/1.1" 200 -
```

For IPv6, since our VPN doesn't give us an IPv6, we enabled logging on the NAT rules and accessed **diagnostic view** on the main-firewall, and saw that the requests are effectively being translated. We tried to make a request to the *pypi.org* site, which has an IPv6.



▶ wan	← 2024-10-14T13:21:45	main-firewall.acme-27.test	mil04s44-in-x0e.1e100.net	ipv6-icmp	let out anything from firewall host itself	0
≡ wan	← 2024-10-14T13:21:45	dnsserver.acme-27.test	mil04s44-in-x0e.1e100.net	ipv6-icmp	nat rule	0

2.11 Task 11 - All the hosts of the ACME network should be able to ping (and receive replies of) the other hosts and the Internet hosts

Initial brainstorming - We have to let the needed traffic flow.

Evaluation of the security policy - Hosts in the ACME network can ping each other and initiate ping requests to external hosts on the internet using both IPv4 and IPv6.

Policy implementation in OPNsense - We enabled ICMP traffic for both IPv4 and IPv6 on all interfaces of the ACME network, except for the WAN interface, where specific and stricter rules are applied, as detailed in the following section.

Test of the configuration - To test the task requirements, we initiated ping requests from various hosts for every possible pair, using both IPv4 and IPv6, and ensured that everything worked correctly. We also pinged our own machine using IPv4; however, it was not possible to test IPv6 with the external host.

2.12 Task 12 - Only hosts in the DMZ should be reachable using the ping and traceroute tools from the Internet

Initial brainstorming - To enforce the policy, we can allow only the necessary traffic on the WAN interface, blocking all other traffic for the remaining subnets at the earliest possible point.

Evaluation of the security policy - This policy must not conflict with the one from the previous point, so only ICMP Echo Requests from the Internet should be blocked, not Echo Replies. Traceroute generally works with ICMP time exceeded packets and UDP packets in the port range of 33434–33534.

Policy implementation in OPNsense - We added two inbound rules on the WAN interface (for IPv4 and IPv6, respectively) to allow ICMP Echo Requests to the DMZ, followed by two additional rules to explicitly block them for the other subnets. We did the same to allow and then block the ICMP Time exceeded and the UDP traffic for the traceroute.

Test of the configuration - We tested ping and performing traceroute from our own host to the hosts in the DMZ and the other subnets, ensuring that the latter were blocked.

⊗ wan	→ 2024-10-13T20:42:27	100.101.0.2:60342	100.100.1.10:33465	udp	block traceroute UDP from WAN to any (but DMZ) ipv4
⊗ wan	→ 2024-10-13T20:42:27	100.101.0.2:57860	100.100.1.10:33464	udp	block traceroute UDP from WAN to any (but DMZ) ipv4
⊗ wan	→ 2024-10-13T20:23:33	100.101.0.2	100.100.4.100	icmp	block ping request from WAN to any (but DMZ) ipv4
⊗ wan	→ 2024-10-13T20:23:32	100.101.0.2	100.100.4.100	icmp	block ping request from WAN to any (but DMZ) ipv4

Since we don't know the IPv6 route to the ACME network, we couldn't perform the test with IPv6.



2.13 Task 13 - ICMP redirect packets should not cross any network

Initial brainstorming - Limiting or blocking ICMP redirects is generally considered a good security practice.

Evaluation of the security policy - ICMP redirects should be blocked everywhere, and hosts should not be allowed to receive or generate them.

Policy implementation in OPNsense - Since the task applies to the entire ACME network, we created the necessary rules directly in the floating rules tab, applying them to all interfaces to block ICMP and ICMPv6 Redirects. Moreover, we modified the following system tunables in the */etc/sysctl.conf* on every host, to avoid the acceptance and the sending of these packets:

```
1 net.ipv4.conf.all.accept_redirects = 0
2 net.ipv4.conf.default.accept_redirects = 0
3 net.ipv4.conf.eth0.accept_redirects = 0
4 net.ipv6.conf.all.accept_redirects = 0
5 net.ipv6.conf.default.accept_redirects = 0
6 net.ipv6.conf.eth0.accept_redirects = 0
7
8 net.ipv4.conf.all.send_redirects = 0
9 net.ipv4.conf.default.send_redirects = 0
10 net.ipv4.conf.eth0.send_redirects = 0
```

We also verified that the corresponding tunables were disabled on the two firewalls as well.

Test of the configuration - To test the configuration in this step, we forged ICMP Redirect packets from our host and from the Kali machine using **hping3** for IPv4 and the **scapy** library for IPv6. All packets were blocked before reaching their destination, and no routing tables were modified.

```
(user@pci)~$ sudo hping3 --icmp --icmp-type 5 --icmp-code 1 --icmp-gw 100.100.4.1 100.100.6.2
[sudo] password for user:
HPING 100.100.6.2 (eth0 100.100.6.2): icmp mode set, 28 headers + 0 data bytes
^C
— 100.100.6.2 hping statistic —
57 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

2.14 Task 14 - Anything that is not explicitly allowed has to be denied

Initial brainstorming - We decided to block most protocols and allow only HTTP, HTTPS, DNS, ICMP, ICMPv6, NTP, and SSH traffic across the entire ACME network.

Evaluation of the security policy - Deny everything not explicitly allowed in the previous rules, and allow only the used protocols.



Policy implementation in OPNsense - By default, OPNsense denies all incoming traffic while allowing everything in egress. Therefore, on the DMZ, EXTERNAL_CLIENTS, INTERNAL_CLIENTS and INTERNAL_SERVERS interfaces, we blocked all outgoing traffic. Between the two firewalls, only the explicitly allowed traffic between the two portions of the ACME network has been permitted.

Test of the configuration - To test the configuration, we carried out tests on all the previous rules to ensure that everything was correctly working.

3 Conclusions

We successfully configured the logserver, the HTTPS redirect on the webserver, a transparent proxy for internal hosts, a forward proxy for all other hosts, and all other requirements. However, testing the security policies using IPv6 proved to be challenging because our VPN does not provide IPv6 support.